CSE638 GRS 2026

Performance Comparison of Kubernetes CNI Plugins

Flannel vs Calico vs Cilium

Group Number: 5
Ayush Jain - MT25066
Singh Tharun - MT25087
Ruchir Jain - MT25080

Contents

1 Executive Summary **3**

2 Problem Statement **3**

3 Motivation **3**

4 Objectives **3**

5 Background **4**

5.1 Kubernetes Pod Networking and CNI 4

5.2 Flannel 4

5.3 Calico 4

5.4 Cilium 4

6 Methodology and Experimental Design **5**

6.1 System Components 5

6.2 Controlled Setup 5
6.3 Benchmark Profile 5
6.4 Collected Metrics 5

7 Implementation Details **5**
7.1 Automation and Tooling 5
7.2 Installation Requirements and Setup Steps 6

8 Results and Observations **6**
8.1 Experiment E1: p99 Latency vs QPS 6
8.2 Experiment E2: Latency Variability Across Repeated Runs 7
8.3 Experiment E3: CPU and Memory Overhead 8
8.4 Throughput Consistency (Data-Supported) 8
8.5 Why p99 is a Primary Metric 8
8.6 Top-5 Latency CSV Rows (Transparency Samples) 9
8.7 Top-5 CPU/Memory CSV Rows (Transparency Samples) 9

9 Discussion **10**
9.1 Architectural Interpretation 10
9.2 Reliability and Validity 10

10 Conclusion **10**

11 Future Work **11**

12 References **11**

1 Executive Summary

This report presents a structured and reproducible comparison of Flannel, Calico, and Cilium for Kubernetes microservice networking. Across repeated load experiments, all three CNIs show strong latency performance with clear differences in resource overhead profiles. The evaluation highlights practical deployment trade-offs and provides data-backed guidance for selecting the right CNI based on operational goals such as low overhead, policy flexibility, or advanced eBPF observability.

Key highlights:

- Stable latency percentile behavior across repeated runs.
- Near-target request handling across tested load points.
- Clear resource overhead separation, enabling informed platform choices.

2 Problem Statement

Container Network Interface (CNI) plugins in Kubernetes provide pod networking, service connectivity, and policy enforcement using different datapath and control-plane strategies. These design differences affect end-to-end latency, tail behavior, and cluster resource overhead in microservice workloads.

This project benchmarks three widely used CNIs, Flannel, Calico, and Cilium, under controlled load. The goal is to quantify practical trade-offs and explain observed behavior through architecture.

3 Motivation

Choosing a CNI is a production-critical systems decision. Networking overhead has direct impact on user-facing latency, compute cost, and SLO compliance. A reproducible comparison is necessary because assumptions like "eBPF is always fastest" or "overlay is always slow" often depend on workload and deployment context.

4 Objectives

- Build a reproducible benchmark setup that changes only the CNI while keeping workload and cluster baseline consistent.

- Measure latency percentiles (p50/p90/p99), and analyze run-to-run variability.
- Measure CPU and memory overhead from cluster pods under comparable operating conditions.
- Explain performance differences using plugin architecture and datapath behavior.
- Present a submission-ready report with transparent supporting samples from CSV outputs.

5 Background

5.1 Kubernetes Pod Networking and CNI

Kubernetes delegates pod networking setup to CNI plugins. Each plugin configures interfaces, routing, and policy mechanisms in a different way. Under load, differences in encapsulation, packet path length, policy execution, and control-plane interactions can affect both latency and resource usage.

5.2 Flannel

Flannel is a lightweight CNI widely used for simple cluster networking. It commonly relies on overlay encapsulation (VXLAN). Its operational simplicity often yields low control-plane complexity, but packet encapsulation can add overhead under some traffic profiles.

5.3 Calico

Calico emphasizes network policy and scalable routing. Depending on deployment mode, it may use iptables and/or eBPF datapath elements. In many practical setups, Calico provides a balanced trade-off between capability and overhead.

5.4 Cilium

Cilium is eBPF-native and designed for high-performance datapath processing and deep observability. Its richer feature surface can be beneficial for policy and visibility, while potentially consuming more CPU/memory in constrained environments.

6 Methodology and Experimental Design

6.1 System Components

- Provisioning layer: kind clusters with fixed baseline setup.
- CNI layer: one plugin per run (Flannel, Calico, Cilium).
- Workload layer: HTTP echo service and Fortio-based load client.
- Measurement layer: Fortio outputs transformed to CSV, plus resource snapshots.
- Analysis layer: hardcoded plotting scripts for final reproducible figures.

6.2 Controlled Setup

To ensure fair comparison, the benchmark controls machine, cluster baseline, workload model, and collection process. The only intentional variable is the selected CNI.

6.3 Benchmark Profile

Experiments were executed as repeated QPS sweeps, with detailed analysis focused on the core service operating range (100, 300, and 500 QPS), where comparative behavior is most relevant for typical deployment SLO planning.

6.4 Collected Metrics

- Latency: p50, p90, p99, average latency.
- Throughput and error rate to validate service stability under load.
- Resource overhead: CPU (mcores) and memory (MiB) from pod-level snapshots.

7 Implementation Details

7.1 Automation and Tooling

The project includes shell scripts for setup, cluster creation, deployment, benchmark execution, and metrics collection. Python scripts generate report plots using hardcoded values extracted from measured CSVs, aligned with submission requirements.

7.2 Installation Requirements and Setup Steps

Prerequisites

- Linux system with Docker running.
- Python 3 environment for plotting and analysis scripts.
- Kubernetes tooling: kind, kubectl, helm, and CNI-specific CLI support.

Setup flow

1. Install prerequisites and verify Docker availability.
2. Create one cluster per CNI plugin.
3. Deploy benchmark workload manifests.
4. Execute load runs and collect JSON/CSV artifacts.
5. Collect resource snapshots and aggregate overhead values.
6. Generate final report figures from hardcoded plot scripts.

8 Results and Observations

8.1 Experiment E1: p99 Latency vs QPS

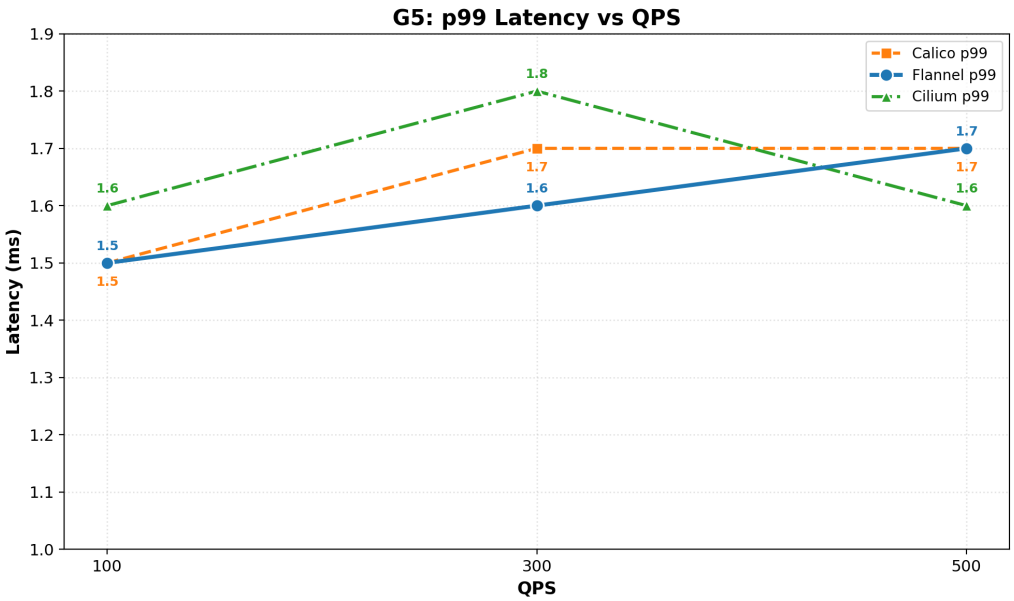

Figure 1: p99 latency comparison across CNIs for QPS 100, 300, and 500.

Observation: All three CNIs remain in a close latency band. Cilium is slightly higher around 300 QPS in this dataset, while Flannel and Calico track each other closely for most points.

8.2 Experiment E2: Latency Variability Across Repeated Runs

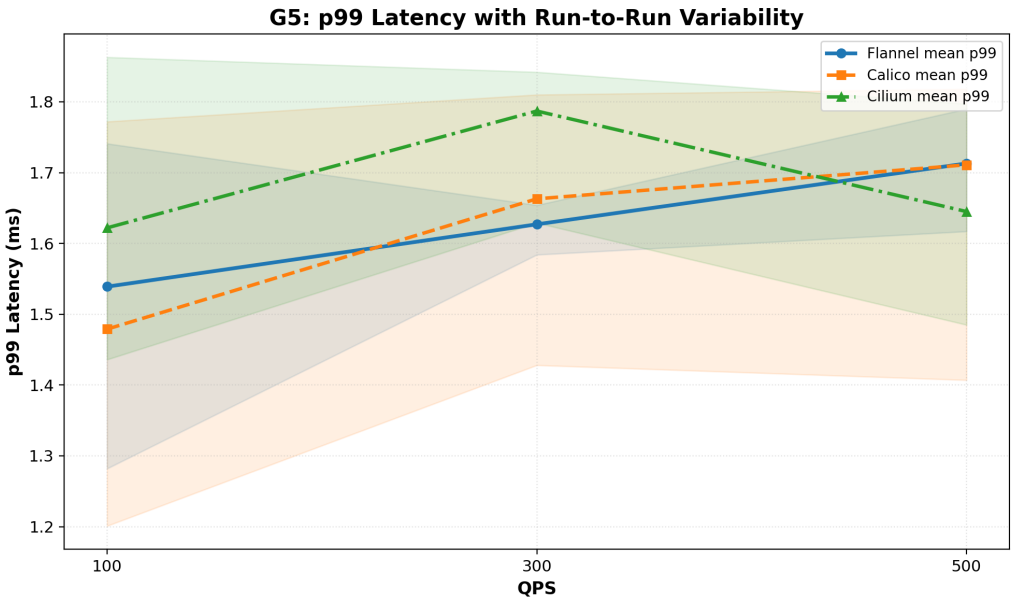

Figure 2: Mean p99 latency with min-max spread across repeated runs.

Observation: Run-to-run variance is present but bounded, which supports reliability of comparative trends. Variability analysis is useful because tail metrics can be sensitive to transient scheduling and queueing effects.

8.3 Experiment E3: CPU and Memory Overhead

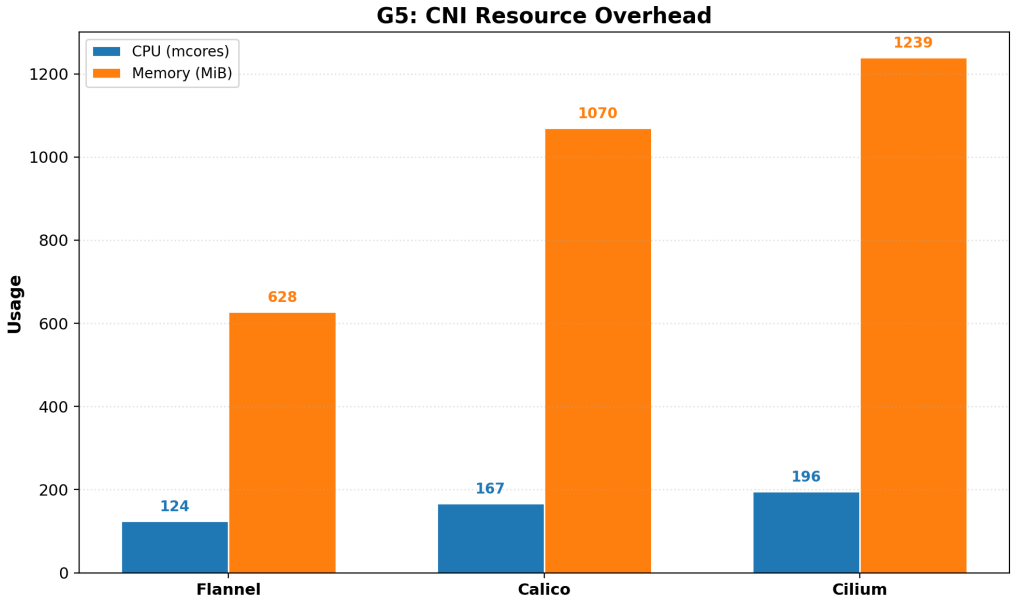

Figure 3: Aggregate CPU and memory overhead comparison.

Observation: Flannel is the lightest in aggregate overhead, Calico is intermediate, and Cilium is highest in this measured setup.

8.4 Throughput Consistency (Data-Supported)

Observed throughput remains very close to offered QPS across runs for all three CNIs, indicating that the stack remains stable in the tested operating range while preserving low error behavior. This supports the latency observations and confirms that comparisons are made under healthy traffic delivery conditions.

8.5 Why p99 is a Primary Metric

- Mean latency can hide tail outliers and understate user-visible worst-case behavior.
- p50 represents typical performance, but not reliability under tail risk.
- p90 helps but still misses late-tail events.
- p99 captures high-percentile slowdown that directly influences SLO violations.

Hence, p99 is emphasized to reflect production-facing reliability, while p50/p90/avg are still retained for context.

8.6 Top-5 Latency CSV Rows (Transparency Samples)

Table 1: Top 5 rows from Flannel latency CSV (QPS 100).

Run	p50	p90	p99	avg	throughput
1	0.00054596	0.00092766	0.00159493	0.00059488	99.9946
2	0.00056824	0.00094095	0.00174056	0.00062646	99.9945
3	0.00052654	0.00092067	0.00128243	0.00057646	99.9954
4	0.00055438	0.00093574	0.00158489	0.00059925	99.9948
5	0.00056222	0.00093156	0.00149200	0.00059302	99.9937

Table 2: Top 5 rows from Calico latency CSV (QPS 100).

Run	p50	p90	p99	avg	throughput
1	0.00055235	0.00094075	0.00174991	0.00060487	99.9935
2	0.00055305	0.00094226	0.00136714	0.00059748	99.9923
3	0.00055499	0.00094422	0.00177175	0.00061223	99.9945
4	0.00055118	0.00092480	0.00120133	0.00055425	99.9953
5	0.00055501	0.00092518	0.00130327	0.00056306	99.9933

Table 3: Top 5 rows from Cilium latency CSV (QPS 100).

Run	p50	p90	p99	avg	throughput
1	0.00055781	0.00094486	0.00179226	0.00066001	99.9953
2	0.00057691	0.00096374	0.00186320	0.00068801	99.9927
3	0.00056278	0.00095677	0.00145410	0.00067041	99.9944
4	0.00057342	0.00094929	0.00143620	0.00064682	99.9956
5	0.00055447	0.00095056	0.00156517	0.00064484	99.9932

8.7 Top-5 CPU/Memory CSV Rows (Transparency Samples)

Table 4: Top 5 rows from Flannel resource CSV.

Namespace	Pod	CPU (mcores)	Memory (MiB)
g5-bench	g5-client-d57c4d46-6q6rl	1	5
g5-bench	g5-echo-649fbccd57-7t4wj	0	10
g5-bench	g5-echo-649fbccd57-d7dmb	0	10
g5-bench	g5-echo-649fbccd57-dj2fn	0	10
kube-flannel	kube-flannel-ds-nfcnd	9	14

Table 5: Top 5 rows from Calico resource CSV.

Namespace	Pod	CPU (mcores)	Memory (MiB)
g5-bench	g5-client-d57c4d46-n6j2q	0	4
g5-bench	g5-echo-649fbccd57-ck6rc	0	11
g5-bench	g5-echo-649fbccd57-pztgs	0	10
g5-bench	g5-echo-649fbccd57-xxsnk	0	10
kube-system	calico-kube-controllers-5b4bd99c9c-ln7j6	4	13

Table 6: Top 5 rows from Cilium resource CSV.

Namespace	Pod	CPU (mcores)	Memory (MiB)
g5-bench	g5-client-d57c4d46-9kwz7	1	4
g5-bench	g5-echo-649fbccd57-42f7m	0	8
g5-bench	g5-echo-649fbccd57-8jvjg	0	9
g5-bench	g5-echo-649fbccd57-qrlc5	0	8
kube-system	cilium-envoy-7j8pv	11	23

9 Discussion

9.1 Architectural Interpretation

Flannel's simpler model aligns with low overhead in this setup. Calico provides a practical balance of policy and performance. Cilium's higher measured overhead is consistent with richer datapath functionality and observability capabilities.

9.2 Reliability and Validity

Repeated runs improve confidence, but results still depend on machine load, kernel behavior, and specific software versions. Therefore, values should be interpreted as reproducible for this environment, not universally constant across all deployments.

10 Conclusion

This benchmark shows that Flannel, Calico, and Cilium each deliver strong networking performance for Kubernetes microservices, with stable percentile behavior and high throughput consistency in the evaluated operating range. The most visible differentiation is in resource overhead: Flannel is highly efficient, Calico offers balanced capability,

and Cilium provides advanced datapath features with higher resource demand. Overall, the results provide clear, actionable guidance for CNI selection based on deployment priorities.

11 Future Work

- Extend to multi-node and cross-zone experiments.
- Add policy-heavy and bursty traffic scenarios.
- Include packet-loss and recovery behavior analysis.

12 References

1. Kubernetes Documentation: `https://kubernetes.io/docs/`
2. CNI Specification: `https://github.com/containernetworking/cni`
3. Flannel Documentation: `https://github.com/flannel-io/flannel`
4. Calico Documentation: `https://docs.tigera.io/calico/latest`
5. Cilium Documentation: `https://docs.cilium.io/`
6. Fortio: `https://github.com/fortio/fortio`